

# Connecting Agile UX With Agile Software Development

## Highlights

- Basics of agile SE (software engineering) methods.
- Lifecycle aspects of agile SE.
- Planning in agile SE.
- Sprints in agile SE.
- Challenges in agile SE from the UX perspective.
- What is needed on the UX side?
- Problems to anticipate.
- A synthesized approach to integrating agile UX.
- Integrating UX into planning.
- Integrating UX into sprints.
- Synchronizing the two agile work flows.

## 29.1 INTRODUCTION

In [Chapter 4](#), we introduced the agile UX funnel model, and in all subsequent process chapters, we have assumed an agile UX process. Now, we discuss some details about how the agile UX process can connect to an agile SE process in real-world project environments.

Agile software engineering (SE) approaches, now well-known and commonly used, are incremental, iterative, and test-driven means of frequently delivering pieces of useful working software to customers. In the past, agile SE approaches have not taken UX into account. And, because traditional UX processes have not fit well within agile SE environments, the UX side has struggled to adjust their methods to fit SE constraints.

### *Agile Software Engineering*

A lifecycle process approach to software implementation based on delivering frequent small-scale operational and usable releases that can be evaluated to yield feedback ([Section 29.2](#)).

### *Funnel Model (of Agile UX)*

A way of envisioning UX design activities before syncing with agile software engineering sprints (for overall conceptual design in the early funnel) and after syncing with software engineering (for individual feature design in the late funnel) ([Section 4.4](#)).

At the end of the day, the entire system development team needs an overall approach that includes agile UX while retaining the basics of agile SE approaches. In this chapter, we present a variation of our agile UX process methods that will integrate well with existing agile SE processes by accounting for the constraints imposed by those agile SE processes.

### 29.1.1.1 *Agility is not (just) about being fast*

Lots of people think the point of an agile approach is to be fast, but that misses the main point. Being agile means being light on your feet, nimble, and responsive to change during the process. Yes, an agile process is also considered fast. Compared to the old waterfall method, agile methods are very fast. But that is just a bonus side effect, not the essence of it. It's about *how* it is fast.

Agility is about being fast by not wasting time on things you don't need or will never use. It's about being fast by knowing what doesn't work before investing time and effort pursuing the wrong course. It's about being fast by focusing on the product or system rather than the process. But, more than anything else, it's about being nimble and responsive to change. A project is not a race; it's an obstacle course, or sometimes even a labyrinth. You can't know what will try to derail you from your course until you encounter it and have to react. In the old slow and careful waterfall method, you never get the feedback you need to even know there is a problem to respond to until the whole project is almost over.

#### *Waterfall Lifecycle Process*

One of the earliest formal software engineering lifecycle processes; an ordered linear sequence of lifecycle activities, each of which flowed into the next like a set of cascading tiers of a waterfall (Section 4.2).

### **Example: Working Too Fast Can Hurt You in the Marketplace**

This is an extension of an example we started in [Chapter 1 \(Section 1.8.3\)](#). A number of years ago I (HRH) consulted for a small company that developed large e-commerce software systems. They had produced a new application in their line of software products, but they were having trouble with customer satisfaction. When we described the UX lifecycle process for understanding users and basing UX design on their needs and work context, we were quickly told there was no time for user research. They proudly mentioned that they were very agile organization and developed new products and systems in “internet time,” working tirelessly to release products on an almost impossible schedule.

This latest system was no exception; they released it to their customers after an unbelievably short lifecycle, but it wasn't very good. But, fortunately, they had a cadre of loyal existing customers who bought essentially everything they released. Not surprisingly, they found the new system difficult to use, but they needed that functionality, so, they invested significant time and effort into learning how to use it. But this new version of the system got bad reviews and a

bad reputation in the marketplace. Their marketing group became quite concerned that, although they sold a few units to existing customers, they were not able to bring in new customers.

So, they panicked, brought it back to the drawing board, and pulled everyone in to work on an improved version, this time applying some principles of UX design. The result was a system that, in fact, was better but quite different from the original version.

How did the marketplace react? Well, potential new customers did not take the bait. The company's reputation was already spoiled. What about the existing customers? Well, they were very angry and upset. They had gone through all the trouble and expense of learning the old system, and now it had changed completely. They now had to learn yet another new system. In the end, the company had managed to alienate almost all the existing and potential customers, and it was a real crisis for them. Therein lies a profound lesson about sacrificing quality for the sake of speed.

### 29.1.2 Don't Practice Agility Blindly

Finally, before we get into the discussion of agile SE methods, a word of caution: Some practitioners of agile software development have adopted it almost as a religion, applying it without considering the consequences and pushing it beyond its useful limits. And there is danger in agile UX practitioners following that trend. The agile process is a tool and shouldn't take on a cult-like life of its own. Use it when it works, but don't stop asking what's the best process for your project.

We begin by reviewing the essence of the agile SE approach and then identify what is needed for the two agile processes to fit together.

---

## 29.2 BASICS OF AGILE SE METHODS

Much of what is said here about agile methods in SE goes back to the fundamental work of [Kent Beck \(2000\)](#), one of the most authoritative sources on agile SE development methods as embodied in the approach called eXtreme Programming (XP).<sup>1</sup> We have taken words from Beck and other authors and tried to blend them into a summary of the practice. Accurate representations are credited to these authors, while errors in representation are on us.

---

<sup>1</sup>There are other "brands" of approaches to agile SE methods beyond XP, including Scrum ([Rising & Janoff, 2000](#)), but for convenience we focus on XP.

### 29.2.1 Goals and Principles of Agile SE

To establish agreement on early agile software methods, a group met at a workshop in Snowbird, Utah in February 2001 and worked out an “agile manifesto”<sup>2</sup> in which the following principles emerged:

- Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.
- Welcome changing requirements, even late in development. Agile processes harness change for the customer’s competitive advantage.
- Deliver working software frequently, from a couple of weeks to a couple of months, with a preference for the shorter timescale.
- Business people and developers must work together daily throughout the project.
- Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.
- The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.
- Working software is the primary measure of progress.
- Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
- Continuous attention to technical excellence and good design enhances agility.
- Simplicity—the art of maximizing the amount of work not done—is essential.
- The best architectures, requirements, and designs emerge from self-organizing teams.
- At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

Also from that manifesto, practitioners of agile SE methods value:

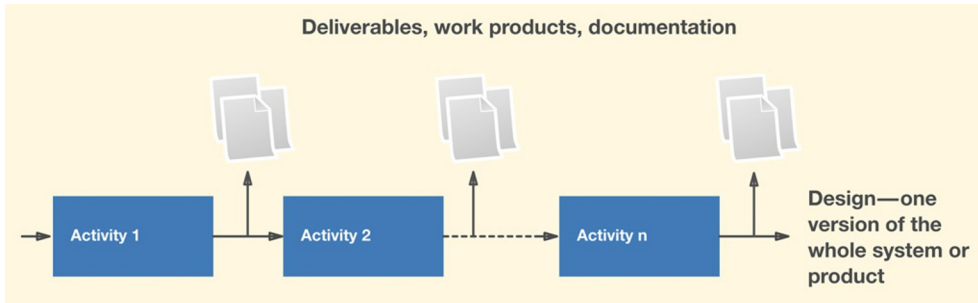
- Individuals and interactions over processes and tools.
- Working software over comprehensive documentation.
- Customer collaboration over contract negotiation.
- Responding to change over following a plan.

### 29.2.2 Contrasting With the Waterfall Method

Such general principles can seem like platitudes, but it wasn’t always so obvious. Perhaps the best way to understand the impact of these principles is through the most extreme contrast, a contrast with the old SE “waterfall” lifecycle development methodology (Royce, 1970). The waterfall model (Fig. 29-1) is a heavy process in the sense that it proceeds slowly and deliberately, finishing all of

---

<sup>2</sup><http://agilemanifesto.org/>.



*Fig. 29-1  
Heavy waterfall process  
with deliverables and  
documentation after every  
process activity.*

one process activity before starting the next, and thereby, not being responsive to change. By the end, needs and requirements have changed, and the product is off course and mostly irrelevant. Resources are wasted.

### 29.2.2.1 Operating in Silos

Each stage of the lifecycle was a complete enterprise in itself, occurring in its own more or less closed environment that came to be known as a silo. In each silo, a team of specialists for the work of that stage worked with its own management and produced its own deliverables. Each stage typically produces documentation, usually voluminous. You didn't get any working version of anything until the very end of the lifecycle, at which point, you got one version of the whole system or product. The problem often was, though, that it took so long to get there that, by the time you arrived, the design and even the basic concept were likely to be out of date. Certainly any requirements that had been painstakingly developed and documented in the requirements phase would no longer be complete or correct. And one long iteration wouldn't have been enough to get the design right, anyway.

#### Silos

Major lifecycle activities in the waterfall model were sometimes called "silos," because they strongly compartmentalized the activities, a compartmentalization that was usually reflected in the developer or contractor's organization (Section 6.6).

### 29.2.3 Characteristics of Agile SE Methods

Agile SE development methods begin coding very early. Agile SE has a shorter, almost nonexistent, requirements engineering phase and far less documentation than that of traditional SE. As typified in XP, agile SE code implementation occurs in small increments and iterations.

Small releases are delivered to the customer after each short iteration, or development cycle. In most cases, these small releases, although limited in functionality, are intended to be working versions of the whole system that run by themselves and deliver some useful capability for the customer.

In simplest terms, agile SE development methods “describe the problem simply in terms of small, distinct pieces, then implement these pieces in successive iterations” (Constantine, 2002, p. 3). Each piece is tested until it works and is then integrated into the rest. Next, the whole is tested in what Constantine calls “regression testing,” until the new feature works with all the previously developed pieces. As a result, the next iteration always starts with something that works.

The agile software development methods are further characterized by a profound emphasis on communication, especially continuous communication with the customer. Informal communication is strongly preferred over formal. Close communication is emphasized to the point that they have an onsite customer as part of the team, giving feedback continuously.

### 29.2.3.1 Avoiding big design upfront

A main principle of agile SE methods is to avoid big design upfront. This means the approach generally eschews upfront ethnographic and field studies and extensive requirements engineering. The idea is to get code written as soon as possible and resolve problems by reacting to customer feedback later.

SE practitioners verify that they are writing the code correctly by the practice of pair programming. Code is written by two programmers working together and sharing one computer and one screen; that is, a colleague is always watching over each programmer’s shoulder.

Of course, pair programming is not new with agile methods. Even outside agile SE methods (and before they existed, pair programming) was a proven technique with a solid track record (Constantine, 2002). Code is further verified by regular and continuous testing against an inventory of test cases.

#### Scope (of Delivery)

Describes how the target system or product is “chunked” (broken into what size pieces) in each iteration or sprint for delivery to the client and users for feedback and to the software engineering team for agile implementation (Section 3.3).

#### Top-Down Design

A UX design approach starting with abstract descriptions of work activities, stripped of information about existing work practice and working toward a best design solution independent of current perspectives and biases (Section 13.4).

## 29.3 LIFECYCLE ASPECTS OF AGILE SE

If this process were to be represented by a lifecycle diagram, it would not be a waterfall or even an iteration of stages, but a set of overlapping microdevelopment activities. In agile approaches developers do just enough—a microlevel of each activity—to support one small feature; Fig. 29-2 illustrates XP as an example agile method.

This shows that building an e-commerce website in the waterfall approach would require listing all requirements that must be supported in the website before starting a top-down design. In an agile approach, the same website would be built as a series of smaller features, such as a shopping cart or checkout module.

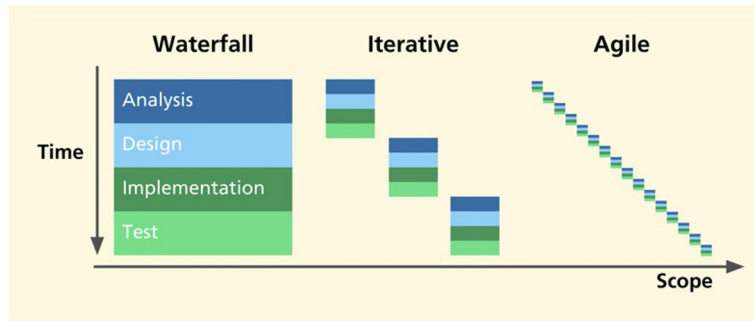


Fig. 29-2

Comparison of scope of development activities across methodologies, taken with permission from Beck (1999, Fig. 1).

### 29.3.1 Planning in Agile SE Methods

In our discussion of how an agile SE method works,<sup>3</sup> we are roughly following XP as a guide. As shown in Fig. 29-3, each iteration consists of two parts: planning and a sprint to implement and test the code for one release.

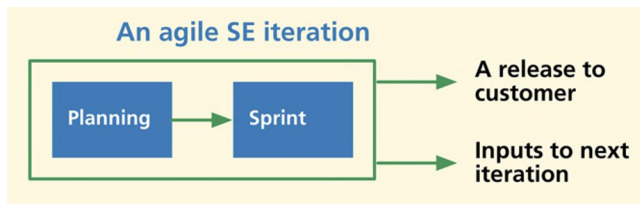


Fig. 29-3

Abstraction of an agile SE release iteration.

#### 29.3.1.1 Customer stories

We have already been discussing user stories since Chapter 7, where the inputs to them are collected, Chapter 8, where the essence of user stories is extracted from the raw usage data, and Chapter 10, where user stories are written. Now, we show how they play a pivotal role in connecting the UX and SE in late funnel development activities.

For the moment, we will use the term “customer story,” which has been used in the context of agile SE because they don’t work with users. As we bring agile UX into this process, we will be switching to “user stories,” which can include both sources.

#### Sprint

A relatively short (not more than a month) time period within an agile software engineering schedule in which a working and releasable product increment is implemented. It is a unit of work being done in an agile SE environment; an iteration associated with a release (to the client and/or users) (Sections 3.3, 29.3.2, and 29.7.2).

<sup>3</sup>This is the way things should work “in theory.” As with most UX and SE methods, there are many variations in practice.

The planning part of each SE iteration in Fig. 29-3 yields a set of customer-written stories, prioritized by business value and cost to implement. A customer story has a role a bit like that of a use case, a scenario, or a requirement. A customer story, written on a story (index) card, is a description of a customer-requested feature. It is a narrative about how the system is supposed to solve a problem, representing a chunk of functionality that is coherent and useful in some way to the customer.

### 29.3.1.2 Story-based planning

Expanding the “planning” box of Fig. 29-3, we get the details of how customer stories are used in SE planning, as shown in Fig. 29-4.

As shown in Fig. 29-4, developers start the planning process by sitting down with onsite customer representatives. They ask customer representatives to think about the most useful chunks of functionality that can add business or enterprise value. The customer writes stories about the need for these pieces of functionality. This is the primary way that the developers understand “requirements,” indirectly through the customer representatives.

Developers assess the stories and estimate the effort required to implement (program) a solution for each, writing the estimate on the story card. Typically, in XP, each story gets a 1-, 2-, or 3-week estimate in “ideal development time.” More on how these estimates are created follows.

The customer sorts and prioritizes the story cards by choosing a small set for which the cost estimates are within a predetermined budget and which represent features they want to include in a “release.” Prioritization might result in lists of stories or requirements labeled as “do first,” “desired—do, if time,” and “deferred—consider next time.” Developers break down the stories into

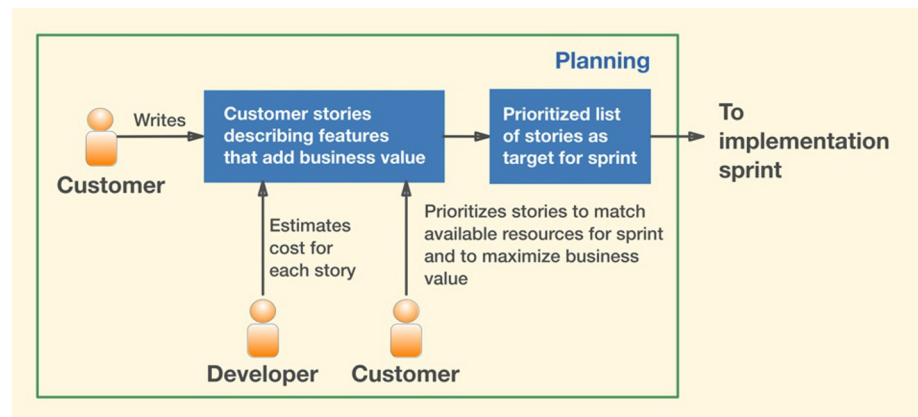


Fig. 29-4  
Customer stories as the basis  
of agile SE planning.



development tasks, each written on a task (for the developers to do) card. Each such development task is giving a point count representing an estimate of the amount of development effort required to implement that task. Then, all tasks in a story card are totaled to arrive at the ideal development time.

The output of the planning box, which goes to the upcoming implementation sprint, is a set of customer-written stories, prioritized by cost to implement.

### 29.3.1.3 *Managing customer stories and development tasks*

The most popular project management system used in industry practice to manage stories and associated tasks is Atlassian's JIRA software.<sup>4</sup> This system provides product managers and SE roles capabilities to plan releases, manage user stories and associated tasks for current and future sprints, acceptance criteria for each user story, and the current status of each story. JIRA also provides a robust defect (bug) tracking capability including ability to plan which bugs will be fixed in a given release, including dependencies and other blockers for resolving those bugs.

### 29.3.1.4 *Controlling scope*

Customer stories are the local currency in what Beck (2000, p. 54) calls the “planning game” through which the customer and the developers negotiate the scope of each release. At the beginning, there is a time and effort “budget” of the person-hours or level of effort available for implementing all the stories, usually per release.

As the customer prioritizes story cards, the total of the work estimates is kept and, when it reaches the budget limit, the developers' “dance card” is full. Later, if the customer wants to “cut in” with another story, they have to decide which existing customer story with an equal or greater value must be removed to make room for the new one. So no one, not even the boss, can just add more features.

This approach gives the customer control of which stories will be implemented, but affords developers a tool to battle scope or feature creep. Developer estimates of effort can be way off, probably in most cases underestimating the effort necessary, but at least it lets them draw a line in the sand. With experience, developers get pretty good at this estimation, called team velocity, given a particular technology platform and application domain. The better developers get at estimation, the more smoothly the entire project will go.

## *Sprint*

A relatively short (not more than a month) time period within an agile software engineering schedule in which a working and releasable product increment is implemented. It is a unit of work being done in an agile SE environment; an iteration associated with a release (to the client and/or users) (Sections 3.3, 29.3.2, and 29.7.2).

<sup>4</sup><https://www.atlassian.com/software/jira>.

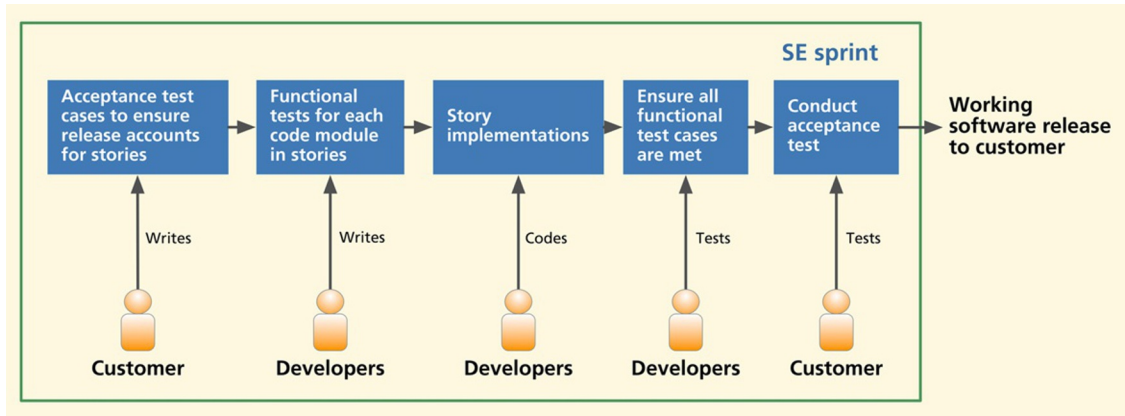


Fig. 29-5  
An agile SE sprint.

## 29.3.2 Sprints in Agile SE Methods

In Fig. 29-5, we show an expansion of the “sprint” box of Fig. 29-3. Each agile SE sprint consists of activities that are described in the following sections.

### 29.3.2.1 Acceptance test creation

The customer writes the functional acceptance tests. There is no process for this, so it can be kind of fuzzy, but it does put the customer in control of acceptance of the eventual code. With experience, customers get good at this. Usually the acceptance criteria on which the acceptance tests are based are summarized in the corresponding customer stories as a list of feature capabilities that the customer has to be able to perform.

### 29.3.2.2 Unit code test creation

The team divides the work by assigning customer stories to be coded in the next sprint. A programmer picks a customer story card and finds a programming partner. Before any coding, together, the pair writes unit tests that can verify that functionality is present in the code that is yet to be written.

### 29.3.2.3 Implementation coding

The programming pairs work together to write the code for modules that support the functionality of selected customer stories. As they work, the partners do on-the-fly design. The agile SE literature says almost nothing about design.

The programmers do not worry about the higher level architecture; the system architecture supposedly evolves with each new slice of functionality added to the overall system. The programming pair integrates this code into the latest version.

#### 29.3.2.4 Code testing

Next, the programming pair runs the unit code tests designed for the modules just implemented to make sure that the realization of this functionality in code is correct.

#### 29.3.2.5 Regression testing

In addition to testing the code for the current features, the team runs all code tests again on all modules coded so far until all tests are passed, to ensure that the new module doesn't break any of the previously implemented modules. This allows developers to make code modifications based on changing requirements, while ensuring that all existing parts of the code continue to function properly.

#### 29.3.2.6 Acceptance testing and deployment

Developers submit this potentially shippable product functionality to the customer for acceptance review. Upon acceptance, the team deploys this small iterative release to the customer.

## 29.4 CHALLENGES OF AGILE SE METHODS FROM THE UX PERSPECTIVE

Agile methods make SE practitioners feel productive and in control because they and their customers, and not some overarching design, drive the process. These methods are inexpensive, fast, and lightweight, with early deliverables. The pair-programming aspect also seems to produce high-reliability code with fewer bugs. Nonetheless, agile SE methods are programming methods developed by programmers for programmers, and they pose some challenges from the UX perspective.

Because big design upfront is shunned, there is no upfront analysis to glean general concepts of the system and associated work practice. The one customer representative on the team is not required even to be a real user and can't be expected to represent all viewpoints, needs and requirements, usage issues, or usage context. There may be no real user data at all upfront, and coding will end up being "based only on assumptions about user needs" (Mommel, Gundelsweiler, and Reiterer, 2007, p. 169). There is no identification of user tasks and no process for identifying user interaction or information needs.

Beyer, Holtzblatt, and Baker (2004) echo this criticism of using a customer representative as the only application domain expert. As they point out, under their Axiom 2, "Make the user the expert," many customer representatives are

### *Big Design Upfront*

A lifecycle process approach that involves starting with ethnographic and field studies and extensive requirements engineering, of which the old Waterfall method is an archetype (Section 29.2.3.1).

### Ecology

In the setting of UX design, the ecology is the entire set of surrounding parts of the world, including networks, other users, devices, and information structures, with which a user, product, or system interacts (Section 16.2.1).

### Top-Down Design

A UX design approach starting with abstract descriptions of work activities, stripped of information about existing work practice and working toward a best design solution independent of current perspectives and biases (Section 13.4).

### Early Funnel

The part of the funnel (agile UX model) for large-scope activity, usually for conceptual design, before syncing with software engineering (Section 4.4.4).

### Late Funnel

The part of the funnel (agile UX model) for small-scope activity, for syncing with agile software engineering sprints (Section 4.4.3).

not also users and, therefore, can't necessarily speak for the work practice of others.

Beyond these predesign drawbacks, the agile SE method does not involve design, so, there is no room in the process for design ideation.

## 29.5 WHAT IS NEEDED ON THE UX SIDE

Nonetheless, in some ways, the UX process lifecycle is a good candidate to fit with agile software methods because it is already iterative. But there is a big difference. The traditional UX lifecycle is built on starting with a complete understanding of users and their needs followed by extensive design activities, both missing in agile SE. Regarding the early agile usage research, [Beyer et al. \(2004\)](#) recommend devoting an iteration of usage research with real users to understand user needs. They say they have been able to get through quick usage research and early design with five to eight users in one to two weeks. (Our experience, even on medium complexity projects has been more in the order of 1–2 months.)

In this section, we discuss the considerations necessary to adjust traditional UX methods to adapt to agile SE approaches ([Mammel et al., 2007](#)).

To work in the UX domain, an agile method must involve some early analysis devoted to understanding user work activities and work context and gleaning general concepts of the system within its ecology. And some early design activities must also be retained in the process to give structure and coherence to how interaction and information fit into the design. And, if top-down design is needed for an innovative product design, it should take place before any involvement on the SE side. Early-funnel UX activities accommodate exactly those needs.

At the same time, to be compatible with the agile SE side of development, UX methods must:

- Be lightweight.
- Emphasize team collaboration and require colocation.
- Include effective customer and user representatives.
- Adjust UX lifecycle activities to be compatible with SE sprint-based incremental releases by switching focus from top-down holistic design to bottom-up feature design.
- Include ways to control scope.

Late-funnel UX activities are designed to accommodate these needs.

## 29.6 PROBLEMS TO ANTICIPATE

In a special-interest-group workshop at CHI 2009 (Miller and Sy, 2009), a group of UX practitioners met to share their experiences in trying to incorporate a user-centered design approach into the agile SE process. Among the difficulties experienced by these practitioners in their own environments were:

- Sprints too short; not enough time for customer contact, design, and evaluation.
- Inadequate opportunities for user feedback, and the user feedback they did get was ignored.
- Customer representative weak, not committed, and lack of colocation.
- No shared vision of broader conceptual design because focus is on details.
- Risk of piecemeal results in a bottom-up approach.

### 29.6.1 UX and SE Don't Always Work Together the Way They are Supposed To

In Fig. 29-6, we show a general picture of how UX and SE are supposed to work together in design and implementation.

In either the traditional or agile lifecycle, the UX team is supposed to provide a UX design as specifications to the SE team to implement and integrate with the corresponding functionality. If everyone on both sides of the project (and management as well) agrees to this simple model at the beginning, the matter of integrating UX and SE within the project will be well on its way to being solved.

In reality, however, the SE team is often well into its process when the UX team is introduced to the project, and the UX team is doomed to play catch-up for the rest of the project, and, of importance to all, the project can never benefit from

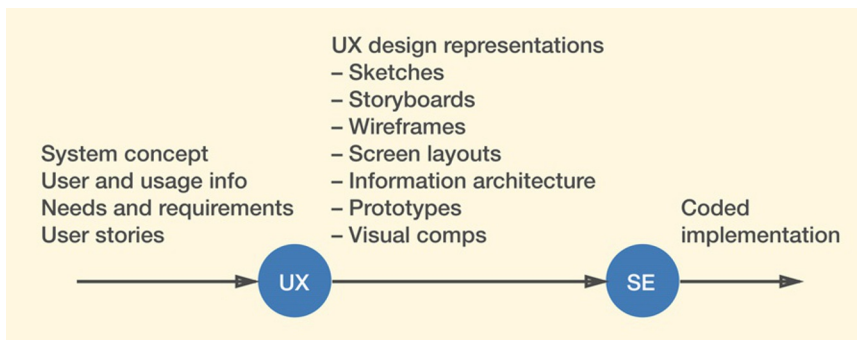


Fig. 29-6  
How UX and SE work together.

what the UX team could have done. In the worst of scenarios, the UX team is called in to review and give feedback on the work done so far by the SE team.

While it is true that the SE team needs its own inputs in addition to those provided by the UX team, this must be done in an open and shared way. Both teams get some of the same kinds of inputs (e.g., customer or user stories) but use them in different ways to produce different kinds of designs. The main requirement is that all inputs obtained from the client or users are shared with both teams. The SE team does not gather requirements or design recommendations without involving the UX team from the beginning.

### 29.6.2 The Need for a Full Overview: The Software Side Versus the UX Side

Because the agile approach to SE is incremental and bottom-up, it precludes the luxury of seeing a full overview of the system upfront. In fact, some agile teams see not having an overview as a “badge” testifying to their correct use of the agile method, not being distracted from their feature-at-a-time emerging view of the system. The whole point of the agile approach is that they can’t afford to establish a system overview upfront, nor do they really need to. That is mainly because the user doesn’t see the software directly.

But building a system a little piece at a time can be risky, at least on the UX side. The user’s view of the system is the user interface. This view, depending directly on the UX design, must look as though it is built around a well-integrated conceptual model, a single unified style or theme, and not a patchwork of different ideas and views. If not, it will lead to a confusing and disjointed user experience. It’s almost impossible to develop a unified user interface exclusively by piecewise and bottom-up methods, one feature at a time. So, the UX team needs an overview to emerge as soon as possible.

---

## 29.7 A SYNTHESIZED APPROACH TO INTEGRATING AGILE UX AND AGILE SE

Most of the early literature about incorporating UX into the agile SE development process is based on either adjusting UX design methods to somehow keep pace with existing agile SE methods or trying to do just selected parts of UX design processes in the presence of an essentially inflexible agile SE method.

While it is possible that XP, for example, and some abbreviated UX design techniques can coexist and work together, in these add-on approaches, the two

parts are not really combined (McInerney and Maurer, 2005; Patton, 2002, 2008). This creates a coping scenario on the UX side, as UX practitioners attempt to live with the constraints while trying to ply their own processes within an overall development environment driven solely by the agile SE method.

Here, we have tried to synthesize an approach to allow our agile UX process to mesh with an agile SE environment without compromising on essential UX needs.

We especially acknowledge the influence of Constantine and Lockwood (2003), Beyer et al. (2004), Meads (2010), and Lynn Miller (2010).

### 29.7.1 Integrating UX into Planning

Fig. 29-7 shows a scheme for integrating the UX role into the planning box of Fig. 29-3.

Planning in Fig. 29-7 features small upfront analysis and design by the UX designer, customer, and users, in the left-most box, producing a broad (if partial) understanding of the work domain, work practice, and conceptual model (next to left-most box). This upfront UX activity occurs in the early agile UX funnel.

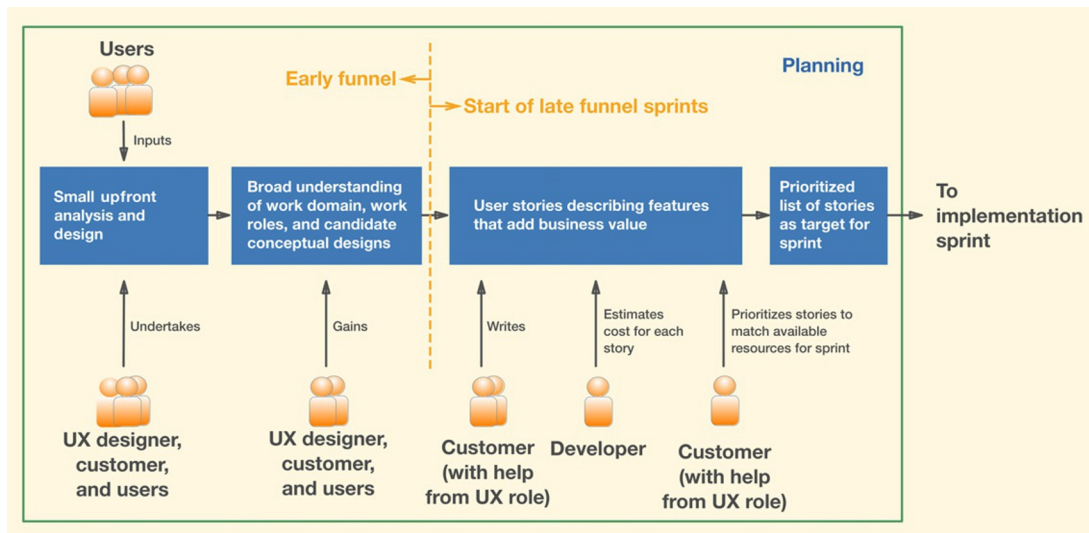


Fig. 29-7

*Integrating the UX role into early agile UX funnel planning.*

### Scope (of Delivery)

Describes how the target system or product is “chunked” (broken into what size pieces) in each iteration or sprint for delivery to the client and users for feedback and to the software engineering team for agile implementation (Section 3.3).

### Early Funnel

The part of the funnel (agile UX model) for large-scope activity, usually for conceptual design, before syncing with software engineering (Section 4.4.4).

#### 29.7.1.1 Small upfront analysis and design

In Section 29.2.3.1, we described how SE practitioners avoid big design upfront, preferring instead to write *some* code as soon as possible and then fix problems by reacting to customer feedback later. In UX, we can’t afford to operate this way. The integrity and consistency of all our designs, including the small-scope task-level designs of the late funnel, depend on first establishing a solid overall conceptual design in the early funnel. But the ground work necessary can’t turn the whole process into a waterfall model. So, it has to be light on its feet, and that’s where the small upfront analysis and design come in.

The small upfront analysis and design, in the left-most box of Fig. 29-7, is our way of including an initial abbreviated form of usage research and design that involves the customer and users. This is exactly what happens in the early agile UX funnel. And much of the “design” in the early funnel small upfront analysis and design is conceptual design to establish the high-level consistency overview.

Also, in planning, the UX person also assists the customer in other responsibilities, such as writing and prioritizing stories. These stories are now called user stories rather than customer stories because their substance came from users in the upfront analysis. Although this begins to change the basic agile pattern, it gives the UX team more traction in bringing UX into the overall process. Other authors (Constantine and Lockwood, 1999; Memmel et al., 2007) have agreed that adding a measure of user and/or task modeling is also a very useful supplement in that same spirit.

Goals of the small upfront analysis and design in the early agile UX funnel include:

- Understand the users’ work and its context.
- Identify key work roles, work activities, and user tasks.
- Model workflow and activities in the existing enterprise and system.
- Forge an initial high-level conceptual design.
- Identify inputs for selected user stories that reflect user needs in the context of their work practice.

These early agile UX funnel small upfront analysis and design activities are what we describe in the early chapters (Part 2) of this edition of the book.

**Data capture.** Agile usage research can be as brief or as lengthy as desired or afforded. We suggest interviewing and observing the work practice of at least one or two people in each key work role. There is no recording and no transcript of interviews. The UX practitioners write notes by hand directly on small index cards—a Constantine hallmark—to discourage verbosity in the notes.



***Aim toward effective user stories.*** We are looking for user stories to drive our small pieces of interaction design and prototyping. But what kind of user stories do we seek? Stories about work activities, roles, and tasks can still be a good way for designers to start. However, as Meads (2010) says, users are not interested in tasks per se, but are more interested in features. Following his advice, *we focus on features*, which can entail a substantial set of related user work activities and tasks within a work context. Therefore, we usually come up with a *set* of related user stories for a feature (for example, tasks to create, search for, display, modify, and delete subscriptions to a certain kind of service).

### 29.7.1.2 UX role helps customer write user stories

In the third box from the left in Fig. 29-7, the UX person helps the customer write user stories. As we said earlier, the UX person usually takes charge of user story writing to ensure completeness, appropriate scope, consistency, and cohesion.

Because UX team members, customers, and users participate in the early agile UX funnel small upfront analysis and design, user story writing will be easier, faster, and more representative of real user needs. The UX role influences the customer toward creating stories based on workflows revealed in the agile usage research part of small upfront analysis and design.

### 29.7.1.3 The truth about user stories

As we explained in Section 10.2.1, you can't depend on stories written by just users to be accurate, complete, or comprehensive. So, in real projects, UX analysts and designers take inputs from users collected in usage research and write small-scope user capability requirements as user stories, as though users had expressed them, so the whole collection has a uniform format. This is one important way process variations exist in practice.

### 29.7.1.4 UX role helps customer prioritize user stories

By helping the customer representative prioritize the user stories, the UX person can keep an eye on the overarching vision of user experience and a cohesive conceptual design, thereby steering the result toward an effective set of stories for an iteration.

## 29.7.2 Integrating UX into Sprints

In Fig. 29-8, we show UX counterpart activities occurring in the late funnel during an agile SE sprint of Fig. 29-5.

### Sprint

A relatively short (not more than a month) time period within an agile software engineering schedule in which a working and releasable product increment is implemented. It is a unit of work being done in an agile SE environment; an iteration associated with a release (to the client and/or users) (Sections 3.3, 29.3.2, and 29.7.2).

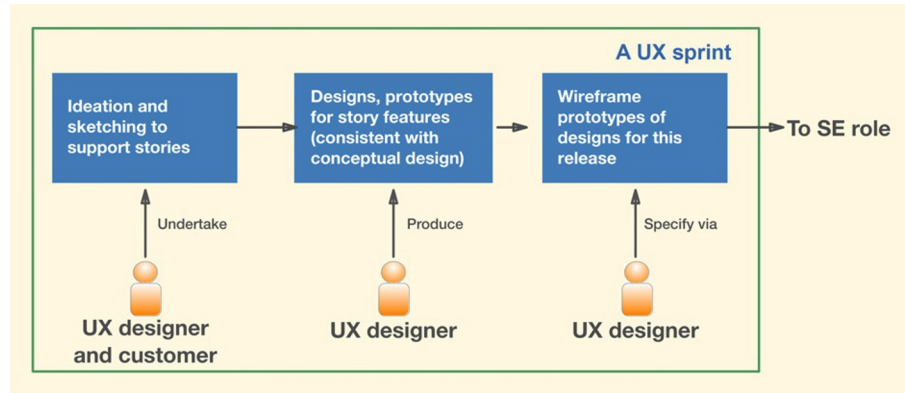


Fig. 29-8

UX counterpart of an agile SE sprint.

While the SE people are doing a sprint, the UX person and customer perform their own version of a sprint, which is shown in Fig. 29-8. They begin by picking a story or picking a feature with a set of related user stories.

With the conceptual design in mind, the UX designers start ideation and sketching of an interaction design to support the functionality of the user story. Initial static wireframe prototypes are used as sketches of interaction designs, leading to “interactive” click-through wireframe prototypes to demonstrate navigation within interaction sequences.

**Design reviews, design walk-throughs, plus iterative design fixes.** At this small scope, the tightly-coupled design creation-prototyping-evaluation cycle will often start with frequent multiple static wireframe sketches. These sketches are iteratively reviewed within the UX team for ideation in task-level design creation. This early design ideation and sketching can also involve brief focused returns to usage research to fill gaps and answer questions about the feature.

When the UX team is satisfied they have their best initial design, it is demonstrated in the form of a click-through wireframe prototype in a walk-through review with all stakeholders, including client/customers and users. The typical result is possibly multiple rounds of feedback, refinement, and further evaluation.

Then, the same kind of walk-through evaluation is done with SE developers, to get feedback on feasibility, consistency, platform issues, etc. This leads to possibly multiple rounds of fixing and refining, leading to a handoff to developers for implementation. By this time, UX designers and software developers should be on the same page about the feature.

After implementation, the UX designer should check the result against the UX designs for fidelity of implementation. Then, the whole team should submit the working feature to the customer for acceptance review.

Table 29-1 shows a mini-UX check-list developed while Hartson was working as a consultant at the Network Infrastructure and Services group at Virginia Tech.

### 29.7.3 Synchronizing the Two Agile Workflows

After describing agile SE planning, agile SE sprints, and UX integration in the late funnel, we now talk about how the UX and SE teams work together and synchronize the workflow in their respective parts of the agile process.

#### 29.7.3.1 Dove-tailed work activities

Miller (2010) proposed a “staggered” approach to parallel track agile development that featured a “criss-cross” interplay between UX activities and SE activities across multiple cycles of agile development. As Patton (2008) put in his blog, the overall approach is characterized as “work ahead, follow behind.”

As Patton says, UX people on agile teams “become masters of development time travel, nimbly moving back and forth through past, present, and future development work.” Based roughly on Miller’s idea, and adding our early funnel concept, we show a scheme in Fig. 29-9 for how UX people and SE people can synchronize their work via a dovetail alternation of activities across progressive iterations.

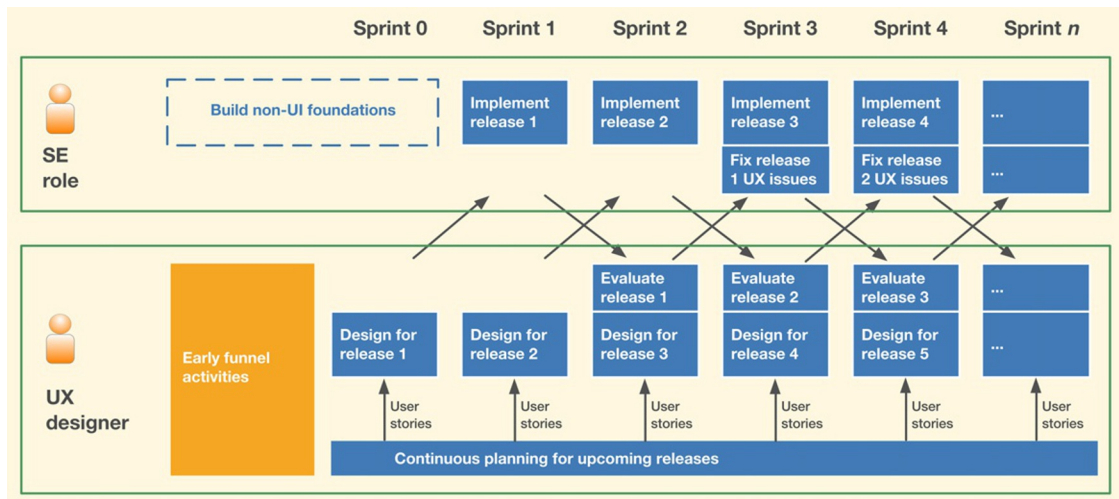


Fig. 29-9

*Alternating UX and SE workflow in an agile process.*

Table 29-1  
*Late-funnel mini-UX process checklist*

| Activity Step                                                                                                                                                                     | Date Performed | Person |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|--------|
| View user stories in JIRA ( <a href="#">Section 29.3.1.3</a> )                                                                                                                    |                |        |
| Task-level usage research and modeling: Identify work roles, HTI (hierarchical task inventory) context, task sequences, business rules, relevant existing apps/screens, work flow |                |        |
| As needed: Define terminology, database schemas                                                                                                                                   |                |        |
| Ideation and sketching of initial UX design                                                                                                                                       |                |        |
| Draw state diagram of flow, if appropriate                                                                                                                                        |                |        |
| Draw wireframes in Sketch                                                                                                                                                         |                |        |
| Connect wireframes with navigation links in InVision and Craft                                                                                                                    |                |        |
| Add link in JIRA to user story to wireframe prototype                                                                                                                             |                |        |
| With UX group: Design review & walkthrough, iterate                                                                                                                               |                |        |
| Get UX agreement                                                                                                                                                                  |                |        |
| With DEV: Initial (preimplementation) design review                                                                                                                               |                |        |
| Update wireframe prototype with changes from DEV                                                                                                                                  |                |        |
| Increase prototype fidelity, if useful                                                                                                                                            |                |        |
| With non-UX stakeholders: Design review, iterate as needed                                                                                                                        |                |        |
| With DEV: Design review and handoff                                                                                                                                               |                |        |
| UX: Create JIRA issue, add link to InVision, assign issue to DEV                                                                                                                  |                |        |
| DEV: Create JIRA issue, assign issue to UX (dependency link to original issue)                                                                                                    |                |        |
| QA: Implementation fidelity check (UX responsibility)—What is built should reflect wireframe prototype exactly                                                                    |                |        |
| If no discrepancy, sign off (UX)                                                                                                                                                  |                |        |
| Else: UX create JIRA issued "Update software with UX review feedback"                                                                                                             |                |        |
| Repeat until no discrepancy                                                                                                                                                       |                |        |

In the original agile SE approach, SE people started first with Sprint 1, taking a set of stories and building a release. That worked when the only thing that was happening was implementation. Now that we are bringing in UX design and evaluation into the mix, we need a few changes, starting with the early funnel activities before going on to initial Sprint 0 on the UX side.

Sprint 0 follows early funnel work on usage research and conceptual design on the UX side and is for beginning the synchronization with the SE cadence. The amount of time these early funnel activities will take depends on the size and complexity of the product or system and the familiarity of the UX team with the problem domain. At the end of Sprint 0, the UX team should have UX designs for the initial feature ready to give to the SE people for their Sprint 1. During this time, the SE people focus on other things, such as building the software infrastructure and services required to support the whole system, what Miller calls building the “high-development, low-UI features.” When the UX people are done with their designs for Release 1, they hand them off to SE people for implementation in Sprint 1, which includes implementation of both functional stories and interaction design components for that cycle.

Not all UX design challenges are equal; sometimes, there is not enough time to address UX design adequately in a given sprint. Then, that design will have to evolve over the design and evaluation activities of more than one sprint. Also, sometimes in interaction design, we want to try out two variations because we are not sure, so that will have to take place over multiple sprints.

Because of the staggering or dovetailing of activities, people on each part of the team are typically working on multiple things at a time. For example, right after handing off the designs for Release 1 to the SE people, the UX people start on designs for Release 2 and continue doing this until the end of Sprint 1 (while the SE people are coding release 1). In any given sprint, say, Sprint  $n$ , the UX people are performing usage research and planning for Sprint  $n + 2$ , while doing design (and prototyping) for Sprint  $n + 1$ , and evaluation of the design for Sprint  $n - 1$ .

Following the “lifecycle” of a single release, Release  $n$ , we see that in Sprint  $n - 1$ , the UX role designs for Release  $n$ , to be implemented by the SE people in Sprint  $n$ . UX evaluates Release  $n$  in Sprint  $n + 1$ . SE fixes it in Sprint  $n + 2$  and rereleases it at the end of that sprint.

### 29.7.3.2 *The value of early delivery*

As [Mammel et al. \(2007\)](#) say, you have the potential to deliver design visions extremely early in the process, resulting in amazingly early involvement of the customer.

Feedback for the first feature can go far beyond just that one feature and its usage. This is the first opportunity for the team to get *any* real feedback. Many additional things can come out of that, not specific to that feature. For example, you will get feedback on how the process is working. You will get feedback on the overall style of your design. You will hear other questions and issues that customers are thinking about that you would not have access to until much later in a nonagile process. Your customers may even reprioritize the story cards based on this interaction. This early feedback fits well with the agile principles of abundant communication and the expectation of early and frequent change.

### 29.7.3.3 *Continuous delivery*

Delivery to customers and users is continuous but in pieces. At the end of any given sprint (call it Sprint  $n$ ), the customer sees a UX prototype of the upcoming release and in the next sprint, Sprint  $n + 1$ , they see the full functional implementation of that prototype. In Sprint  $n + 2$ , they see UX evaluation findings for that same prototype, and, in Sprint  $n + 3$ , they see the final redesign. Each of these points in time is an opportunity for the customer to give feedback on the interaction design.

### 29.7.3.4 *The importance of regression testing*

Regression testing is a step in the agile SE process used to integrate the most recent tested-and-passed feature with all those that have come before it. Without regression testing, the agile method is little more than an incremental waterfall method. You are stuck with decisions made in Sprints 0 and 1, with no ability to iterate major concepts. Unfortunately, time pressures of sprints in real practice can cause teams to short cut regression testing.

Furthermore, this kind of quick testing is easy, and for the most part, automated on the SE side. On the UX side it is a whole different story. Doing regression testing on the UX side would entail running an evaluation after each sprint and using all measuring instruments (e.g., benchmark tasks or surveys) from all previous sprints plus those for the current sprint before releasing it to the customer. In practice, there is almost never enough time to do this.

### 29.7.3.5 *Planning across iterations*

Fig. 29-9 shows planning in a single box at the bottom, continuously extending across all sprint cycles. That is to convey the idea that planning does not occur in discrete little boxes over time at a specific spot in the flow. Planning is more of an “umbrella” activity, distributed over time, and it is cumulative in that the process builds up a “knowledge base” founded on agile usage research

with users. The planning process does not start over for the planning of each cycle.

Instead, the same knowledge base is consulted, updated, and massaged, working with the original small upfront analysis and design results and anything added to supplement those results. Because an overview and conceptual design are evolving in the process, this kind of UX planning brings some top-down benefits to an otherwise exclusively bottom-up process.

#### **29.7.3.6 Communication during synchronization**

This kind of interwoven development process brings with it the risk of falling apart if anything goes wrong. This intensifies the need for constant communication so that everyone remains aware of what everyone else is doing, what progress is being made by others, and what problems are being encountered.

Agile processes can be more fragile than their heavyweight counterparts. Because each part depends on the others in a tightly orchestrated overall activity, if something goes wrong in one place, there might be little time to react to surprises. Constant communication can help minimize this risk.